



第8章

数字逻辑电路基础

8.1 数制与BCD码

8.1.1 常用数制

8.1.2 几种简单的编码

8.2 逻辑代数基础

8.2.1 基本逻辑运算

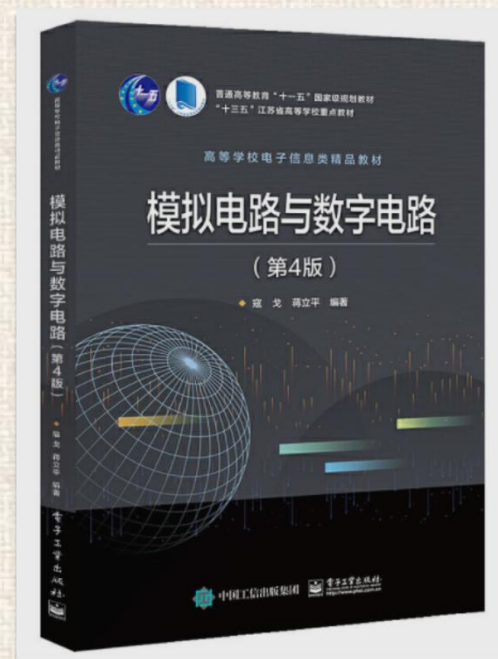
8.2.2 复合逻辑运算

8.2.3 逻辑电平与正、负逻辑

8.2.4 基本定律和规则

8.2.5 逻辑函数的标准形式

8.2.6 逻辑函数的化简





什么是数字逻辑电路？

两类信号：模拟信号；数字信号。

在时间上和幅值上均连续的信号称为模拟信号；

在时间上和幅值上均离散的信号称为数字信号。

处理数字信号的电路称为数字电路。





数字电路特点:

- 1) 工作信号是二进制表示的二值信号(具有“0”和“1”两种取值);
- 2) 电路中器件工作于“开”和“关”两种状态, 电路的输出和输入为逻辑关系;
- 3) 电路既能进行“代数”运算, 也能进行“逻辑”运算;
- 4) 电路工作可靠, 精度高, 抗干扰性好.





8.1 数制与BCD码

所谓“数制”，指进位计数制，即用进位的方法来计数。

数制包括**计数符号（数码）**和**进位规则**两个方面。

常用数制有十进制、十二进制、十六进制、六十进制等。





8.1.1 常用数制

1. 十进制

(1) 计数符号：0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

(2) 进位规则：逢十进一.

例： $1987.45 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 7 \times 10^0$
 $+ 4 \times 10^{-1} + 5 \times 10^{-2}$

(3) 十进制数按权展开式





$$(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i$$

系数 $\xrightarrow{\quad}$ $\xleftarrow{\quad}$ 权

2. 二进制

(1) 计数符号：0, 1 .

(2) 进位规则：逢二进一.

(3) 二进制数按权展开式

$$(N)_2 = \sum_{i=-m}^{n-1} a_i \times 2^i$$





数字电路中采用二进制的原因:

- 1) 数字装置简单可靠;
- 2) 二进制数运算规则简单;
- 3) 数字电路既可以进行算术运算, 也可以进行逻辑运算.

3. 十六进制和八进制

十六进制数计数符号: 0, 1, ., 9, A, B, C, D, E, F.

十六进制数进位规则: 逢十六进一.

按权展开式:

$$(N)_{16} = \sum_{i=-m}^{n-1} a_i \times 16^i$$





例:

$$\begin{aligned}(6D.4B)_{16} &= 6 \times 16^1 + D \times 16^0 + 4 \times 16^{-1} + B \times 16^{-2} \\ &= 6 \times 16^1 + 13 \times 16^0 + 4 \times 16^{-1} + 11 \times 16^{-2}\end{aligned}$$

八进制数计数符号: 0, 1, . . . 6, 7.

八进制数进位规则: 逢八进一.

按权展开式:

$$(N)_8 = \sum_{i=-m}^{n-1} a_i \times 8^i$$





例: $(63.45)_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} + 5 \times 8^{-2}$

4. 二进制数与十进制数之间的转换

(1) 二进制数转换为十进制数(按权展开法)

例: $(1011.101)_2 = 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3}$

$= 8 + 2 + 1 + 0.5 + 0.125$





(2) 十进制数转换为二进制数(提取2的幂法)

例: $(45.5)_{10} = 32 + 8 + 4 + 1 + 0.5$

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1}$$
$$= (101101.1)_2$$

- 数制转换还可以采用**基数连乘、连除**等方法.





5. 二进制算术运算

(1) 二进制算术运算特点

算术运算： 1： 和十进制算数运算的规则相同
2： 逢二进一

特点： 加、减、乘、除全部可以用移位和相加这两种操作实现。

电路结构简化

所以数字电路中普遍采用二进制算数运算





(2) 反码、补码和补码运算

——带符号二进制数的表示及运算

计算机中的符号数可表示为：

符号位+真值



机器数

“0”-----→ 表示正

“1”-----→ 表示负





符号数的表示

- **原码**: 真值 X 的原码记为 $[X]_{\text{真}}$, 在原码表示法中不论数的正负, 数值部分均保持原真值不变。

• *True*

- **反码**: 真值 X 的反码记为 $[X]_{\text{反}}$ 。正数的反码同原码。负数的反码的数值部分为真值的各位按位取反。

• *1's complement*

- **补码**: 真值 X 的补码记为 $[X]_{\text{补}}$ 。正数的补码同原码。负数的补码的数值部分为真值的各位按位取反加1。

• *2's complement*





原码:

- 最高位为符号位，用“0”表示正，用“1”表示负；其余为真值部分
- **优点：** 真值和其原码表示之间的对应关系简单，容易理解
- **缺点：** 计算机中用原码进行加减运算比较困难，0的表示不唯一

8位数0的原码: $+0=0$ 00000000

$-0=1$ 00000000





反码:

对一个机器数 X :

- 若 $X > 0$, 则 $[X]_{\text{反}} = [X]_{\text{原}}$
- 若 $X < 0$, 则 $[X]_{\text{反}}$ = 对应原码的符号位不变, 数值部分按位求反

[例]:

$$X = -52 = -0110100$$

$$[X]_{\text{原}} = \underline{1} \ 0110100$$

$$[X]_{\text{反}} = \underline{1} \ 1001011$$

$$[+0]_{\text{反}} = 00000000$$

$$[-0]_{\text{反}} = 11111111$$

即: 数0的反码也不唯一





补码:

若 $X > 0$, 则 $[X]_{\text{补}} = [X]_{\text{反}} = [X]_{\text{原}}$

若 $X < 0$, 则 $[X]_{\text{补}} = [X]_{\text{反}} + 1$

[例]:

$$X = -52 = -0110100$$

$$[X]_{\text{原}} = 10110100$$

$$[X]_{\text{反}} = 11001011$$

$$[X]_{\text{补}} = [X]_{\text{反}} + 1 = 11001100$$





0的补码：表示唯一

$$[+0]_{\text{补}} = [+0]_{\text{原}} = 00000000$$

$$[-0]_{\text{补}} = [-0]_{\text{反}} + 1 = 11111111 + 1$$

$$= 1 \ 00000000$$

对8位字长，进位被舍掉





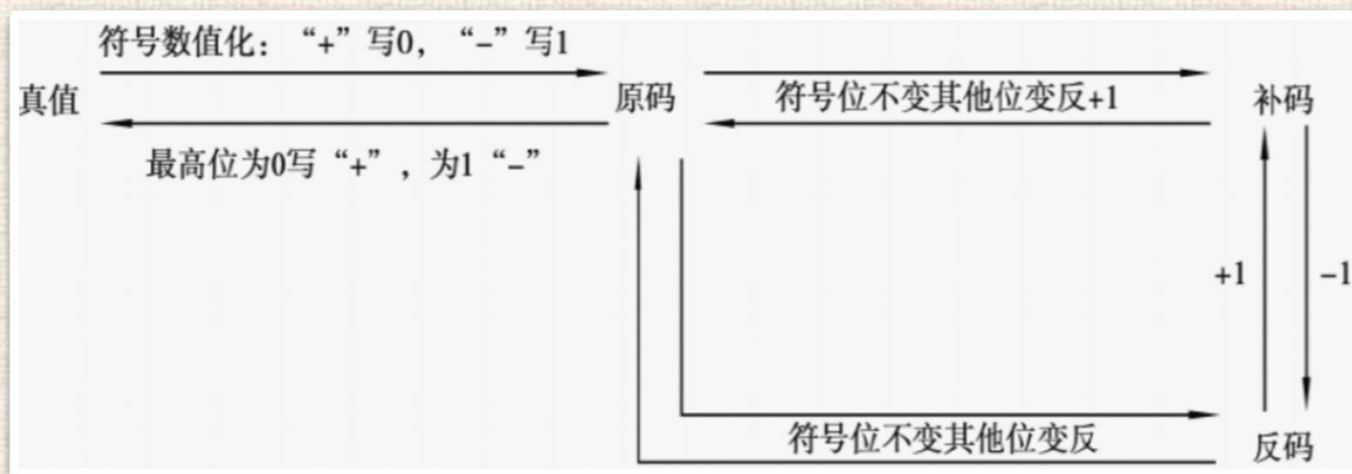
特殊数10000000

- 该数在**原码**中定义为: -0
- 在**反码**中定义为: -127
- 在**补码**中定义为: -128
- 对无符号数, $(10000000)_B = 128$





负数三种编码之间的转换关系





无符号数的表示范围

$$0 \leq X \leq 2^n - 1$$

若运算结果超出这个范围，则产生溢出。

无符号数的溢出判断准则：运算时，当最高位向更高位有进位（或借位）时则产生溢出。





无符号二进制数的运算

算术的四种基本运算：加、减、乘、除

试计算011与010之和

$$\begin{array}{r} 011 \\ + 010 \\ \hline 101 \end{array}$$

结论：两个二进制数相加是通过逐位相加来实现的。





加法运算

推广:

设两个二进制数分别为 $A=A_3A_2A_1A_0$, $B=B_3B_2B_1B_0$ 两数之和为 $S=S_3S_2S_1S_0$

$$S_0=A_0+B_0$$

进位 C_1

$$S_1=A_1+B_1+C_1$$

进位 C_2

$$S_2=A_2+B_2+C_2$$

进位 C_3

$$S_3=A_3+B_3+C_3$$

进位 C_4

$$A+B=C_4S_3S_2S_1S_0$$





减法运算

原理：将减数 B 变成补码后，再与被减数 A 相加，其和（如有进位的话，则舍去进位）就是两个数之差。

什么是补码？

$$\text{补码} = \text{反码（原码取反）} + 1$$

$$\begin{aligned} Y &= 1\ 0\ 0\ 0 - 0\ 1\ 0\ 0 \\ &= 1\ 0\ 0\ 0 + (1\ 0\ 1\ 1 + 1) \\ &= 1\ 0\ 0\ 0 + 1\ 1\ 0\ 0 \\ &= 1\ 0\ 1\ 0\ 0 \quad \text{1 是进位，舍去} \end{aligned}$$





乘法运算

- 对二进制数，乘以2相当于左移一位

[例]：

$$00001011 \times 010 = 0010110B$$

- 方法：
- 1 按照十进制的乘法过程
 - 2 采用移位加的方法





除法运算

- 对二进制数，除以2则相当于右移1位

[例]:

$$00001011 \div 010 = 00000010B$$

即：商=00000010B

余数=11B





带符号数的表示范围:

对8位二进制数:

- 原码: $-127 \sim +127$
- 反码: $-127 \sim +127$
- 补码: $-128 \sim +127$

符号二进制数与十进制的转换

对用补码表示的二进制数转换成十进制:

- 1) 求出真值
- 2) 进行二——十转换





符号数的算术运算

- 通过引进补码，可将减法运算转换为加法运算

$$\text{即: } [X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

$$\begin{aligned} [X-Y]_{\text{补}} &= [X+(-Y)]_{\text{补}} \\ &= [X]_{\text{补}} + [-Y]_{\text{补}} \end{aligned}$$





[例]:

$X = -0110100$, $Y = +1110100$, 求 $X + Y = ?$

- **$[X]_{\text{原}} = 10110100$** 无论正负，真值不变
- **$[X]_{\text{补}} = [X]_{\text{反}} + 1 = 11001100$**
- **$[Y]_{\text{补}} = [Y]_{\text{原}} = 01110100$**
- 所以: **$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$**
 $= 11001100 + 01110100$
 $= 01000000$





两个补码表示的二进制数相加时的符号位讨论

例：用二进制补码运算求出

$13+10$ 、 $13-10$ 、 $-13+10$ 、 $-13-10$

解：

$$\begin{array}{r} +13 \quad 0 \quad 01101 \\ +10 \quad 0 \quad 01010 \\ \hline +23 \quad 0 \quad 10111 \end{array}$$

$$\begin{array}{r} +13 \quad 0 \quad 01101 \\ -10 \quad 1 \quad 10110 \\ \hline +3 \quad 0 \quad 00011 \end{array}$$

$$\begin{array}{r} -13 \quad 1 \quad 10011 \\ +10 \quad 0 \quad 01010 \\ \hline -3 \quad 1 \quad 11101 \end{array}$$

$$\begin{array}{r} -13 \quad 1 \quad 10011 \\ -10 \quad 1 \quad 10110 \\ \hline -23 \quad 1 \quad 01001 \end{array}$$

结论：将两个加数的符号位和来自最高位数字位的进位相加，结果就是和的符号





8.1.2 几种简单的编码

1. 二 - 十进制码 (BCD码) (Binary Coded Decimal codes)

用四位二进制代码来表示一位十进制数码, 这样的代码称为二-十进制码, 或BCD码.

四位二进制有16种不同的组合, 可以在这16种代码中任选10种表示十进制数的10个不同符号, 选择方法很多. 选择方法不同, 就能得到不同的编码形式.

常见的BCD码有8421码、5421码、2421码、余3码等。



常用BCD码

十进制数	8421码	5421码	2421码	余3码
0	0000	0000	0000	0011
1	0001	0001	0001	0100
2	0010	0010	0010	0101
3	0011	0011	0011	0110
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1001
7	0111	1010	1101	1010
8	1000	1011	1110	1011
9	1001	1100	1111	1100





(1) **有权BCD码**：每位数码都有确定的位权的码，

例如：8421码、5421码、2421码。

如：5421码1011代表 $5+0+2+1=8$ ；

2421码1100代表 $2+4+0+0=6$ 。

* 5421BCD码和2421BCD码不唯一。

例：2421BCD码0110也可表示6

* 在表中：

① 8421BCD码和代表0~9的二进制数一一对应；





② **5421BCD码**的前5个码和**8421BCD码**相同，后5个码在前5个码的基础上加1000构成，这样的码，前5个码和后5个码一一对应相同，仅高位不同；

③ **2421BCD码**的前5个码和**8421BCD码**相同，后5个码以中心对称取反，这样的码称为**自反代码**。

例：

4 → 0100	5 → 1011
0 → 0000	9 → 1111





(2) **无权BCD码**: 每位数码无确定的位权, 例如: 余3码.

余3码的编码规律为: 在8421BCD码上加0011,

例 6的余3码为: $0110 + 0011 = 1001$

2. 格雷码(Gray码)

格雷码为无权码, 特点为: 相邻两个代码之间仅有一位不同, 其余各位均相同. 具有这种特点的代码称为**循环码**, 格雷码是**循环码**.





格雷码和四位二进制码之间的关系：

设四位二进制码为 $B_3B_2B_1B_0$ ，格雷码为 $R_3R_2R_1R_0$ ，

则

$$R_3=B_3,$$

$$R_2=B_3\oplus B_2$$

$$R_1=B_2\oplus B_1$$

$$R_0=B_1\oplus B_0$$

其中， $\oplus$ 为异或运算符，其运算规则为：若两运算数相同，结果为“0”；两运算数不同，结果为“1”。





计算机字符集

- 各种文字和符号的总称，包括各国家文字、标点符号、图形符号、数字等。
- 常见字符集名称
 - △ ASCII字符集
 - △ GB2312字符集
 - △ BIG5字符集
 - △ GB18030字符集
 - △ Unicode字符集





ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位	ASCII控制字符											ASCII打印字符													
	0000					0001					0010	0011	0100	0101	0100	0111									
	0					1					2	3	4	5	6	7									
低四位	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0 空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH	标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK	肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	●	^G	BEL	la 响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◼	^H	BS	\b 退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	◯	^I	HT	\t 横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◼	^J	LF	\n 换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v 纵向制表	27	←	^[	ESC	le	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f 换页	28	└	^\	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	\r 回车	29	↔	^]	GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO	移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	🔊	^O	SI	移入	31	▼	^-	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	△	^Backspace 代码: DEL

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。



8.2 逻辑代数基础

研究数字电路的基础为**逻辑代数**，由英国数学家**George Boole**在1847年提出的，逻辑代数也称**布尔代数**。





8.2.1 基本逻辑运算

在逻辑代数中, 变量常用字母 $A, B, C, \dots, Y, Z, a, b, c, \dots, x, y, z$ 等表示, 变量的取值只能是 “0” 或 “1”。

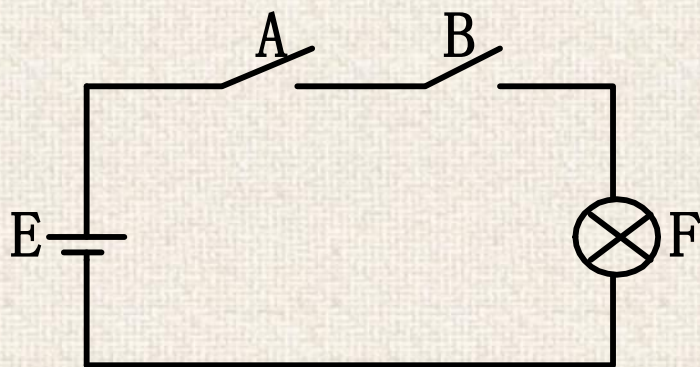
逻辑代数中只有三种基本逻辑运算, 即 “与”、“或”、“非”。





1. 与逻辑运算

定义：只有决定一事件的**全部**条件都具备时，这件事才成立；如果有一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“**与**”逻辑关系。



与逻辑电路

与逻辑电路状态表

开关A状态	开关 B状态	灯F状态
断	断	灭
断	合	灭
合	断	灭
合	合	亮

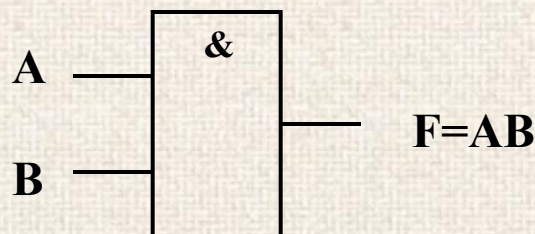




若将开关断开和灯的熄灭状态用逻辑量“0”表示;将开关合上和灯亮的状态用逻辑量“1”表示, 则上述状态表可表示为:

与逻辑真值表

A	B	$F=A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1



与门逻辑符号

与门的逻辑功能概括:

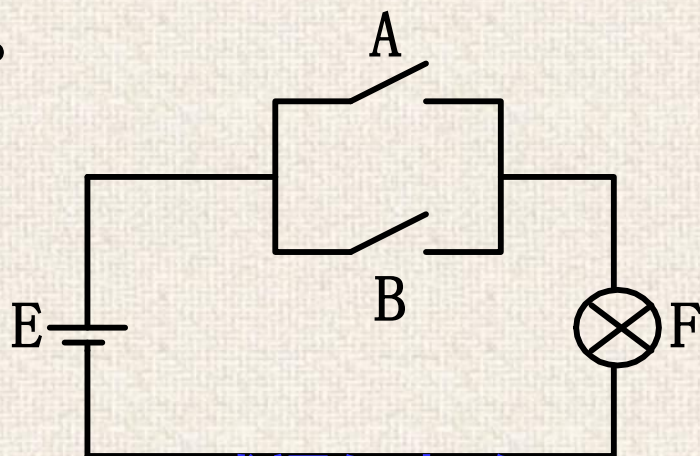
- 1) 有“0”出“0”;
- 2) 全“1”出“1”。





2. 或逻辑运算

定义：在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；只有所有的条件都不具备时，这件事就不成立。这样的因果关系称为“或”逻辑关系。

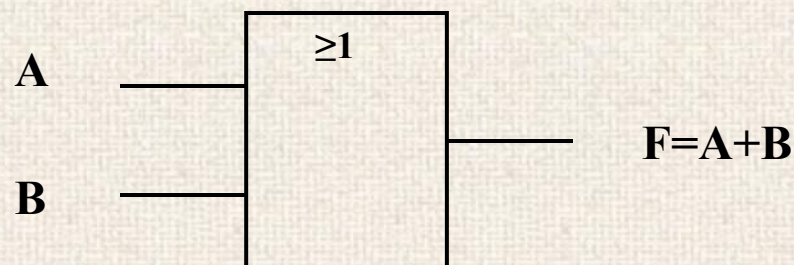


或逻辑电路

或逻辑真值表

A	B	$F=A+B$
0	0	0
0	1	1
1	0	1
1	1	1





或门逻辑符号

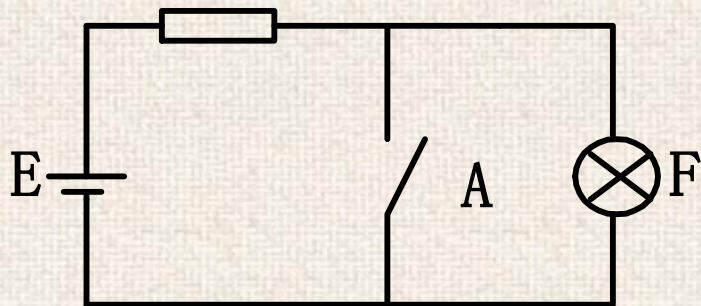
或门的逻辑功能概括为：

- 1) 有“1”出“1”；
- 2) 全“0”出“0”。

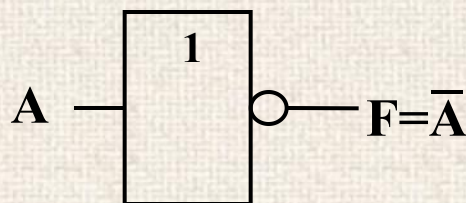
3. 非逻辑运算

定义：假定事件F成立与否同条件A的具备与否有关，若A具备，则F不成立；若A不具备，则F成立. F和A之间的这种因果关系称为“非”逻辑关系。





非逻辑电路



非门逻辑符号

非逻辑真值表

A	$F = \bar{A}$
0	1
1	0

- 与门和或门均可以有多个输入端.



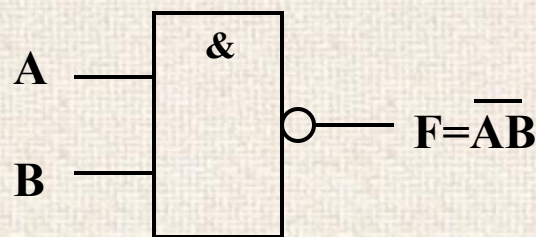


8.2.2 复合逻辑运算

1. 与非逻辑 (将与逻辑和非逻辑组合而成)

与非逻辑真值表

A	B	$F = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0



与非门逻辑符号

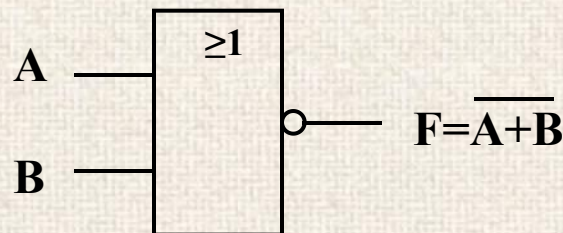




2. 或非逻辑 (将或逻辑和非逻辑组合而成)

或非逻辑真值表

A	B	$F = \overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0



或非门逻辑符号



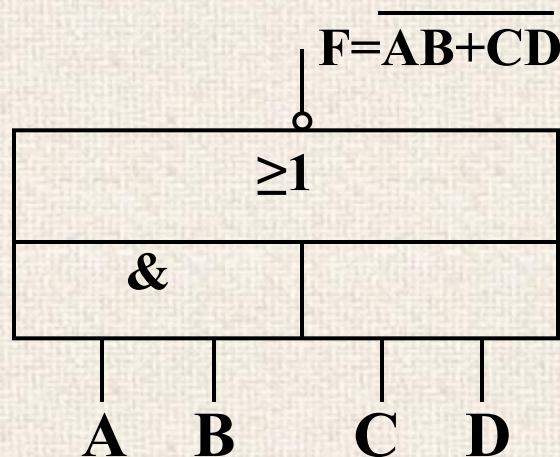


3. 与或非逻辑 (由与、或、非三种逻辑组合而成)

与或非逻辑函数式:

$$F = \overline{AB + CD}$$

与或非门的逻辑符号 →



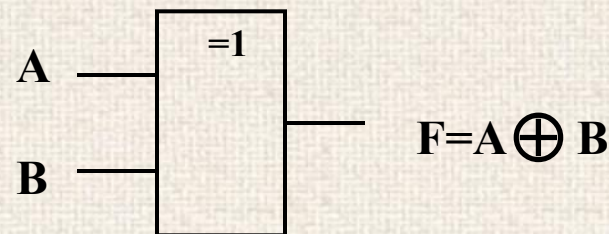


4. 异或逻辑

异或逻辑的函数式为: $F = \overline{A}B + A\overline{B} = A \oplus B$

异或逻辑真值表

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



异或门逻辑符号

异或逻辑的功能为:

- 1) 相同得“0”;
- 2) 相异得“1”.



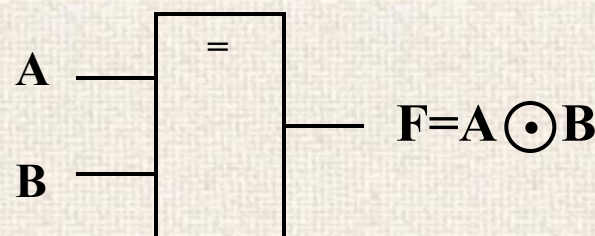


5. 同或逻辑

同或逻辑式为: $F = \overline{A} \overline{B} + A B = A \odot B$

同或逻辑 真值表

A	B	$F = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1



同或门逻辑符号

对照异或和同或逻辑真值表, 可以发现: 同或和异或互为反函数, 即:

$$A \oplus B = \overline{A \odot B}$$





- 表8.13给出了门电路的几种表示方法，本课程中，均采用“**国标**”。国外流行的电路符号常见于外文书籍中，特别在我国引进的一些计算机辅助分析和设计软件中，常使用这些符号。





8.2.3 逻辑电平及正、负逻辑

门电路的输入、输出为二值信号,用“0”和“1”表示.这里的“0”、“1”一般用两个不同电平值来表示.

若用高电平 V_H 表示逻辑“1”,用低电平 V_L 表示逻辑“0”,则称为正逻辑约定,简称正逻辑;

若用高电平 V_H 表示逻辑“0”,用低电平 V_L 表示逻辑“1”,则称为负逻辑约定,简称负逻辑.





在本课程中, 如不作特殊说明, 一般都采用**正**逻辑表示.

V_H 和 V_L 的具体值, 由所使用的集成电路品种以及所加电源电压而定, 有两种常用的集成电路:

1) **TTL**电路, 电源电压为**5**伏, V_H 约为**3****V**左右, V_L 约为**0.2**伏左右;

2) **CMOS**电路, 电源电压范围较宽, **CMOS4000**系列的电源电压 V_{DD} 为**3~18**伏. **CMOS**电路的 V_H 约为**0.9** V_{DD} , 而 V_L 约为**0**伏左右.





对一个特定的逻辑门, 采用不同的逻辑表示时, 其门的名称也就不同.

正负逻辑转换举例

电平真值表			正逻辑(与非门)			负逻辑(或非门)		
V_{i1}	V_{i2}	V_o	A	B	Y	A	B	Y
V_L	V_L	V_H	0	0	1	1	1	0
V_L	V_H	V_H	0	1	1	1	0	0
V_H	V_L	V_H	1	0	1	0	1	0
V_H	V_H	V_L	1	1	0	0	0	1





8.2.4 基本定律和规则

1. 逻辑函数的相等

设有两个逻辑： $F_1=f_1(A_1,A_2,\dots,A_n)$

$$F_2=f_2(A_1,A_2,\dots,A_n)$$

如果对于 $A_1,A_2,\dots,A_n$ 的任何一组取值(共 2^n 组),

F_1 和 F_2 均相等, 则称 F_1 和 F_2 相等.

因此, 如两个函数的真值表相等, 则这两个函数一定相等.



2. 基本定律

① 0—1律	$A \cdot 0 = 0$	;	$A + 1 = 1$
② 自等律	$A \cdot 1 = A$	;	$A + 0 = A$
③ 重迭律	$A \cdot A = A$	;	$A + A = A$
④ 互补律	$A \cdot \overline{A} = 0$	;	$A + \overline{A} = 1$
⑤ 交换律	$A \cdot B = B \cdot A$	;	$A + B = B + A$
⑥ 结合律	$A(BC) = (AB)C$	;	$A + (B + C) = (A + B) + C$
⑦ 分配律	$A(B + C) = AB + AC$	;	$A + BC = (A + B)(A + C)$
⑧ 反演律	$\overline{A + B} = \overline{A} \cdot \overline{B}$	;	$\overline{AB} = \overline{A} + \overline{B}$
⑨ 还原律	$\overline{\overline{A}} = A$		

PLAY





反演律也称**德·摩根**定理, 是一个非常有用的定理.

3. 逻辑代数的三条规则

(1) **代入**规则

任何一个含有变量**x**的等式, 如果将所有出现**x**的位置, 都用一个逻辑函数式**F**代替, 则等式仍然成立.





例： 已知等式 $\overline{A+B}=\overline{A} \cdot \overline{B}$,有函数式 $F=B+C$, 则
用F代替等式中的B,

有
$$\overline{A+(B+C)}=\overline{A} \overline{B+C}$$

即
$$\overline{A+B+C}=\overline{A} \overline{B} \overline{C}$$

由此可以证明反演定律对n变量仍然成立.

(2) 反演规则





设F为任意逻辑表达式, 若将F中**所有**运算符、**常量**及**变量**作如下变换:

·	+	0	1	原变量	反变量
↓	↓	↓	↓	↓	↓
+	·	1	0	反变量	原变量

则所得新的逻辑式即为F的反函数, 记为 **$\bar{F}$** 。

例 已知 $F = \bar{A} B + A \bar{B}$, 根据上述规则可得:

$$\bar{F} = (A + \bar{B})(\bar{A} + B)$$





例 已知 $F=A+B+\overline{C}+\overline{D}+\overline{\overline{E}}$, 则

$$\overline{F}=\overline{A} \overline{\overline{B}} \overline{C} \overline{\overline{D}} \overline{\overline{E}}$$

由F求反函数注意:

- 1) 保持原式运算的优先次序;
- 2) 原式中的不属于单变量上的非号不变;





(3) 对偶规则

设F为任意逻辑表达式, 若将F中所有运算符和常量作如下变换:

·	+	0	1
↓	↓	↓	↓
+	·	1	0

则所得新的逻辑表达式即为F的对偶式, 记为**F'**.

例 有 $F = A \bar{B} + C \bar{D}$

$$F' = (A + \bar{B})(C + \bar{D})$$

例 有 $F = A + \overline{B + \overline{C + D + E}}$

$$F' = A \bar{B} \bar{C} \bar{D} \bar{E}$$





对偶是相互的, F 和 F' 互为对偶式. 求对偶式注意:

- 1) 保持原式运算的优先次序;
- 2) 原式中的长短“非”号不变;
- 3) 单变量的对偶式为自己。

对偶规则: 若有两个逻辑表达式 F 和 G 相等, 则各自的对偶式 F' 和 G' 也相等。

使用对偶规则可使得某些表达式的证明更加方便。

例 :

$$\text{已知 } A(B+C)=AB+AC \xrightarrow{\text{对偶关系}} A+BC=(A+B)(A+C)$$



4. 常用公式

1) 消去律

$$AB + A\bar{B} = A$$

证明:

$$AB + A\bar{B} = A(B + \bar{B}) = A \cdot 1 = A \xrightarrow{\text{对偶关系}} (A+B)(A+\bar{B}) = A$$

2) 吸收律1

$$A + AB = A$$

证明:

$$A + AB = A(1 + B) = A \cdot 1 = A \xrightarrow{\text{对偶关系}} A(A+B) = A$$





3) 吸收律2

$$A + \bar{A}B = A + B$$

证明:

$$A + \bar{A}B = (A + \bar{A})(A + B) = 1 \cdot (A + B) \xrightarrow{\text{对偶关系}} A(\bar{A} + B) = AB = A + B$$

4) 包含律

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

证明:





$$AB + \bar{A}C + BC$$

$$= AB + \bar{A}C + (A + \bar{A})BC$$

$$= AB + \bar{A}C + ABC + \bar{A}BC$$

$$= AB(1 + C) + \bar{A}C(1 + B)$$

$$= AB + \bar{A}C$$

$$\begin{array}{l} \text{对偶关系} \\ \longrightarrow \end{array} \begin{array}{l} (A+B)(\bar{A}+C)(B+C) \\ = (A+B)(\bar{A}+C) \end{array}$$

5) 关于异或和同或运算

对偶数个变量而言,

$$\text{有 } A_1 \oplus A_2 \oplus \dots \oplus A_n = \overline{A_1 \odot A_2 \odot \dots \odot A_n}$$

对奇数个变量而言,

$$\text{有 } A_1 \oplus A_2 \oplus \dots \oplus A_n = A_1 \odot A_2 \odot \dots \odot A_n$$





异或和同或的其他性质：

$$A \oplus 0 = A$$

$$A \oplus 1 = \bar{A}$$

$$A \oplus A = 0$$

$$A \oplus (B \oplus C) = (A \oplus B) \oplus C$$

$$A(B \oplus C) = AB \oplus AC$$

$$A \odot 1 = A$$

$$A \odot 0 = \bar{A}$$

$$A \odot A = 1$$

$$A \odot (B \odot C) = (A \odot B) \odot C$$

$$A + (B \odot C) = (A + B) \odot (A + C)$$

利用异或门可实现数字信号的极性控制.

同或功能由异或门实现.





8.2.5 逻辑函数的标准形式

1. 函数的“与-或”式和“或-与”式

“与-或”式，指一个函数表达式中包含若干个“与”项，这些“与”项的“或”表示这个函数。

例： $F(A,B,C,D)=A+B\bar{C}+AB\bar{C}\bar{D}$

“或-与”式，指一个函数表达式中包含若干个“或”项，这些“或”项的“与”表示这个函数。

例： $F(A,B,C,D)=(A+C+D)(\bar{B}+D)(A+\bar{B}+\bar{D})$





2. 逻辑函数的两种标准形式

1) 最小项的概念

1) 最小项特点 最小项是“与”项。

① n 个变量构成的每个最小项，一定是包含 n 个因子的乘积项；

② 在各个最小项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。





例 有A、B两变量的最小项共有四项(2^2):

$$\bar{A}\bar{B} \quad \bar{A}B \quad A\bar{B} \quad AB$$

例 有A、B、C三变量的最小项共有八项(2^3):

$$\bar{A}\bar{B}\bar{C}, \bar{A}\bar{B}C, \bar{A}B\bar{C}, \bar{A}BC, A\bar{B}\bar{C}, A\bar{B}C, AB\bar{C}, ABC$$

(2) 最小项编号

任一个最小项用 m_i 表示, m表示最小项, 下标 i

为使该最小项为1的变量取值所对应的等效十进制数。





例：有最小项 $A\bar{B}C$, 要使该最小项为1, A、B、C的取值应为1、0、1, 二进制数 101所等效的十进制数为5, 所以 $A\bar{B}C = m_5$

(3) 最小项的性质

- ① 变量任取一组值, 仅有一个最小项为1, 其他最小项为零;
- ② n 变量的全体最小项之和为1;





③ 不同的最小项相**与**，结果为0；

④ 两最小项**相邻**，相邻最小项相“**或**”，可以合并成一项，并可以消去一个变量因子。

相邻的概念： 两最小项如仅有一个变量因子不同，其他变量均相同，则称这两个最小项**相邻**。

相邻最小项相“或”的情况：

例： $A B C + A B \bar{C} = A B$





- 任一 n 变量的最小项，必定和其他 n 个不同最小项相邻。

2) 最大项的概念

(1) 最大项特点 最大项是“或”项。

① n 个变量构成的每个最大项，一定是包含 n 个因子的“或”项；

② 在各个最大项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。





例 有A、B两变量的最大项共有四项：

$$\bar{A} + \bar{B} \quad \bar{A} + B \quad A + \bar{B} \quad A + B$$

例 有A、B、C三变量的最大项共有八项：

$$\bar{A} + \bar{B} + \bar{C}, \bar{A} + \bar{B} + C, \bar{A} + B + \bar{C}, \bar{A} + B + C, \\ A + \bar{B} + \bar{C}, A + \bar{B} + C, A + B + \bar{C}, A + B + C$$

(2) 最大项编号

任一个最大项用 M_i 表示，M表示最大项，下标 i

为使该最大项为0的变量取值所对应的等效十进制数。





例：有最大项 $\bar{A}+B+C$, 要使该最大项为0, A、B、C 的取值应为1、0、0, 二进制数 100所等效的十进制数为4, 所以 $\bar{A}+B+C = M_4$

(3) 最大项的性质

- ① 变量任取一组值, 仅有一个最大项为0, 其它最大项为1;
- ② n变量的全体最大项之积为0;
- ③ 不同的最大项相或, 结果为 1;





④ 两**相邻**的最大项相“**与**”，可以合并成一项，并可以消去一个变量因子。

相邻的概念：两最大项如仅有一个变量因子不同，其他变量均相同，则称这两个最大项**相邻**。

相邻最大项相“**与**”的情况：

例：
$$(A+B+C)(A+B+\overline{C})=A+B$$

- 任一 n 变量的最大项，必定和其他 n 个不同最大项**相邻**。





3) 最小项和最大项的关系

编号下标相同的最小项和最大项互为反函数， 即

$$M_i = \overline{m_i} \quad \text{或} \quad m_i = \overline{M_i}$$

4) 逻辑函数的最小项之和形式

最小项之和式为“与或”式，例：

$$\begin{aligned} F(A,B,C) &= \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC \\ &= \Sigma m(2, 4, 6) \\ &= \Sigma(2, 4, 6) \end{aligned}$$





- 任一逻辑函数都可以表达为最小项之和的形式, 而且是唯一的.

例 : $F(A,B,C) = A B + \bar{A} C$ 该式不是最小项之和形式

$$\begin{aligned} &= AB (\bar{C} + C) + \bar{A} C (\bar{B} + B) \\ &= AB\bar{C} + ABC + \bar{A}\bar{B}C + \bar{A}BC \\ &= \Sigma m (1, 3, 6, 7) \end{aligned}$$

5) 逻辑函数的最大项之积的形式





逻辑函数的最大项之积的形式为“**或与**”式,

例:
$$\begin{aligned} F(A,B,C) &= (A+B+C)(A+\bar{B}+C)(\bar{A}+B+C) \\ &= \Pi M(0, 2, 4) \\ &= \Pi(0, 2, 4) \end{aligned}$$

- **任一**逻辑函数都可以表达为最大项之积的形式, 而且是**唯一**的.





例：
$$\begin{aligned} F(A,B,C) &= (\bar{A} + C)(B + \bar{C}) \\ &= (\bar{A} + B \cdot \bar{B} + C)(A \cdot \bar{A} + B + \bar{C}) \\ &= (\bar{A} + B + C)(\bar{A} + \bar{B} + C)(A + B + \bar{C})(\bar{A} + B + \bar{C}) \\ &= \Pi M(1, 4, 5, 6) \end{aligned}$$

6) 最小项之和的形式和最大项之积的形式之间的关系

若 $F = \sum m_i$ 则 $\bar{F} = \sum_{j \neq i} m_j$

$$F = \overline{\sum_{j \neq i} m_j} = \prod_{j \neq i} \overline{m_j} = \prod_{j \neq i} M_j$$





$$\begin{aligned}\text{例 : } F(A, B, C) &= \Sigma(1, 3, 4, 6, 7) \\ &= \Pi(0, 2, 5)\end{aligned}$$

3. 真值表与逻辑表达式

真值表与逻辑表达式都是表示逻辑函数的方法。

(1) 由逻辑函数式列真值表

由逻辑函数式列真值表可采用三种方法，以例说明：

例： 试列出下列逻辑函数式的真值表。

$$F(A, B, C) = AB + BC$$





方法一： 将A、B、C三变量的所有取值的组合（共八种），分别代入函数式，逐一算出函数值，填入真值表中。

方法二： 先将函数式F表示为最小项之和的形式：

$$\begin{aligned} F(A,B,C) &= AB+BC \\ &= AB (C+\bar{C}) + BC (A+\bar{A}) \\ &= ABC+AB\bar{C}+\bar{A}BC \\ &= \Sigma m (3, 6, 7) \end{aligned}$$





最后根据最小项的性质，在真值表中对应于ABC取值为011、110、111处填“1”，其它位置填“0”。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1





方法三：根据函数式F的含义，直接填表。

函数 $F=AB+BC$ 表示的含义为：

- 1) 当A和B同时为“1”（即 $AB=1$ ）时， $F=1$
- 2) 当B和C同时为“1”（即 $BC=1$ ）时， $F=1$
- 3) 当不满足上面两种情况时， $F=0$





A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

方法三是一种较好的方法，要熟练掌握。





例: $F = \overline{(A \oplus B)(B \oplus C)}$

令: $F_1 = (A \oplus B)$; $F_2 = (B \oplus C)$
 $\overline{F} = F_1 F_2$

A	B	C	F_1	F_2	$\overline{F}$	F
0	0	0	0	0	0	1
0	0	1	0	1	0	1
0	1	0	1	1	1	0
0	1	1	1	0	0	1
1	0	0	1	0	0	1
1	0	1	1	1	1	0
1	1	0	0	1	0	1
1	1	1	0	0	0	1

(2) 由真值表写逻辑函数式





根据最小项的性质，用观察法，可直接从真值表写出函数的最小项之和表达式。

例：已知函数F的真值表如下，求逻辑函数表达式。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1





A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

解：由真值表可见，当ABC取011、101、110、111时，F为“1”。

所以，F由4个最小项组成：

$$\begin{aligned} F(A, B, C) &= \sum m(3, 5, 6, 7) \\ &= \bar{A}BC + A\bar{B}C + ABC\bar{C} + ABC \end{aligned}$$





8.2.6 逻辑函数的化简

化简的意义：

- ① 节省元器件, 降低电路成本;
- ② 提高电路可靠性;
- ③ 减少连线, 制作方便.

逻辑函数的几种常用表达式:





$$\begin{aligned}F(A,B,C) &= A\bar{B} + \bar{A}C \\&= (A+C)(\bar{A}+\bar{B}) \\&= \overline{\overline{A}\overline{B} \cdot \overline{\bar{A}}\overline{\bar{C}}} \\&= \overline{\overline{A}+C + \bar{A}+\bar{B}} \\&= \overline{AB + \bar{A}\bar{C}}\end{aligned}$$

与或式

或与式 (两次求对偶)

与非—与非式 (两次求反)

或非—或非式

与或非式

最简与或表达式的标准:

- 1) 所得与或表达式中, 乘积项 (与项) 数目最少;
- 2) 每个乘积项中所含的变量数最少。





逻辑函数常用的化简方法有：公式法、卡诺图法和列表法。本课程要求掌握公式法和卡诺图法。

1. 公式化简法

针对某一逻辑式, 反复运用逻辑代数公式消去多余的乘积项和每个乘积项中多余的因子, 使函数式符合最简标准.

化简中常用方法:





(1) 并项法

利用公式 $AB + A\bar{B} = A$

例:

$$F = A\bar{B}C + \bar{A}BC + ABC + \bar{A}\bar{B}C$$

在化简中
注意
代入规则
的使用

$$= (A\bar{B} + \bar{A}B)C + (AB + \bar{A}\bar{B})C$$

$$= (A \oplus B)C + (A \odot B)C$$

$$= (A \oplus B)C + \overline{(A \oplus B)}C = C$$

(2) 吸收法

利用公式 $A + AB = A$

例:

$$F = \bar{A} + \overline{A\bar{B}C} \overline{B + AC + \bar{D}} + BC$$

$$= (\bar{A} + BC) + (\bar{A} + BC)\overline{B + AC + \bar{D}}$$

$$= \bar{A} + BC$$

反演律





(3) 消项法 利用公式 $AB + \bar{A}C + BC = AB + AC$

例：

$$\begin{aligned} F &= A\bar{B}C\bar{D} + \bar{A}E + BE + C\bar{D}E \\ &= A\bar{B}C\bar{D} + (\bar{A} + B)E + C\bar{D}E \\ &= A\bar{B}C\bar{D} + \bar{A}\bar{B}E + C\bar{D}E \\ &= A\bar{B}C\bar{D} + (\bar{A} + B)E \\ &= A\bar{B}C\bar{D} + \bar{A}E + BE \end{aligned}$$

(4) 消因子法 利用公式 $A + \bar{A}B = A + B$





例:

$$\begin{aligned} F &= AB + \bar{A}C + \bar{B}C \\ &= AB + (\bar{A} + \bar{B})C \\ &= AB + \bar{A}\bar{B}C \\ &= AB + C \end{aligned}$$

(5) 配项法 利用公式 $\bar{A} + A = 1$; $A \cdot 1 = A$ 等

例:

$$\begin{aligned} F &= AB + \bar{A}C + BC \\ &= AB + \bar{A}C + (A + \bar{A})BC \\ &= AB + \bar{A}C + ABC + \bar{A}BC \\ &= (AB + ABC) + (\bar{A}C + \bar{A}BC) \\ &= AB + \bar{A}C \end{aligned}$$





- 对比较复杂的函数式，要求熟练掌握上述方法，才能把函数化成最简。

2. 卡诺图化简法

该方法是将逻辑函数用一种称为“卡诺图”的图形来表示，然后在卡诺图上进行函数的化简的方法。

1) 卡诺图的构成





卡诺图是一种包含一些小方块的几何图形, 图中每个小方块称为一个单元, 每个单元对应一个最小项. 两个相邻的最小项在卡诺图中也必须是相邻的. 卡诺图中相邻的含义:

- ① 几何相邻性, 即几何位置上相邻, 也就是左右紧挨着或者上下相接;
- ② 对称相邻性, 即图形中对称位置的单元是相邻的.



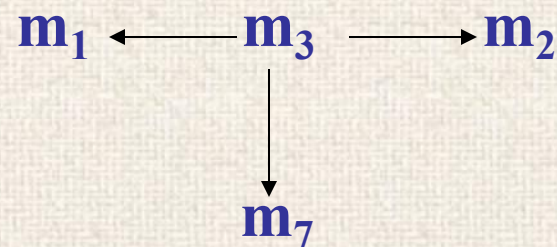


例 三变量卡诺图

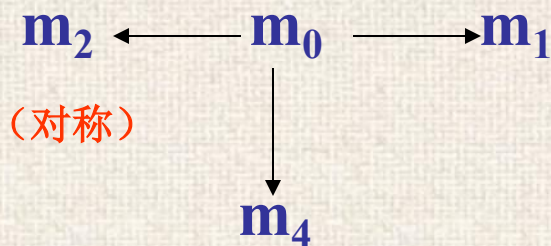
BC \ A	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$ m_0	$\bar{A}\bar{B}C$ m_1	$\bar{A}BC$ m_3	$\bar{A}B\bar{C}$ m_2
1	$A\bar{B}\bar{C}$ m_4	$A\bar{B}C$ m_5	ABC m_7	$AB\bar{C}$ m_6

循环码

相邻性规则



相邻性规则

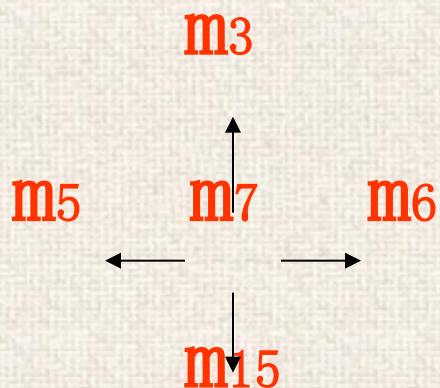




二、四、五变量卡诺图

		B	
		0	1
A	0	0	1
	1	2	3

相邻性规则



		CD			
		00	01	11	10
AB	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10





AB \ CDE									
		000	001	011	010	110	111	101	100
AB	00	0	1	3	2	6	7	5	4
	01	8	9	11	10	14	15	13	12
	11	24	25	27	26	30	31	29	28
	10	16	17	19	18	22	23	21	20

2) 逻辑函数的卡诺图表示法





用卡诺图表示逻辑函数，只是把各组变量值所对应的逻辑函数F的值，填在对应的小方格中。

(其实卡诺图是真值表的另一种画法)

例: $F(A, B, C) = \bar{A}BC + A\bar{B}C + ABC$

用卡诺图表示为:

BC \ A	00	01	11	10
0	0	0	1 _{m₃}	0
1	0	1 _{m₅}	1 _{m₇}	0





3) 在卡诺图上合并最小项的规则

当卡诺图中有最小项相邻时（即：有标1的方格相邻），可利用最小项相邻的性质，对最小项合并。

规则为：

- (1) 卡诺图上任何两个标1的方格相邻，可以合为1项，并可消去1个变量。





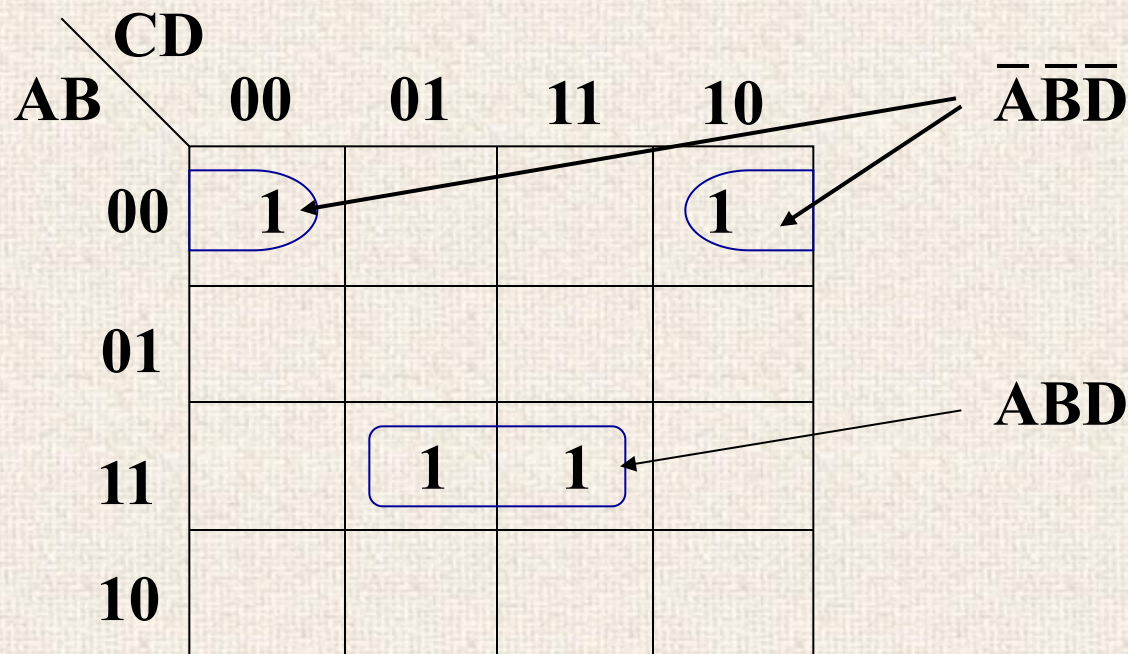
例:

A \ BC	00		01	11	10
	0	0	0	1	0
1	0	1	1	1	0

$$\bar{A}BC + ABC = BC$$

$$A\bar{B}C + ABC = AC$$





(2) 卡诺图上任何四个标1方格相邻，可合并为一项，并可消去两个变量。

四个标1方格相邻的特点：

①同在一行或一列；

②同在一田字格中。





例:

AB \ CD	$\bar{A}\bar{B}$			
	00	01	11	10
00	1	1	1	1
01			1	
11			1	
10			1	

Diagram illustrating a Karnaugh map for a 4-variable function. The map shows a group of four 1s in the first row (AB=00) and a group of four 1s in the third column (CD=11). The groups are labeled $\bar{A}\bar{B}$ and CD respectively.

同在一行或一列

同在一个田字格中

AB \ CD	$\bar{B}\bar{D}$			
	00	01	11	10
00	1			1
01		1	1	
11		1	1	
10	1			1

Diagram illustrating a Karnaugh map for a 4-variable function. The map shows a group of four 1s in the center (AB=01, 11; CD=01, 11) and four 1s at the corners (AB=00, 10; CD=00, 10). The groups are labeled $\bar{B}\bar{D}$ and BD respectively.





思考题

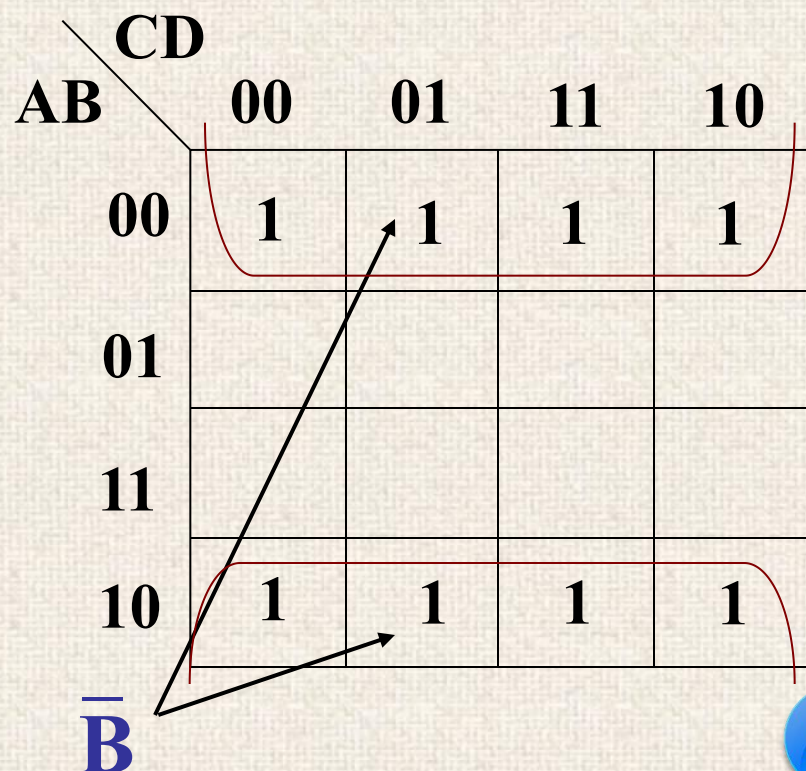
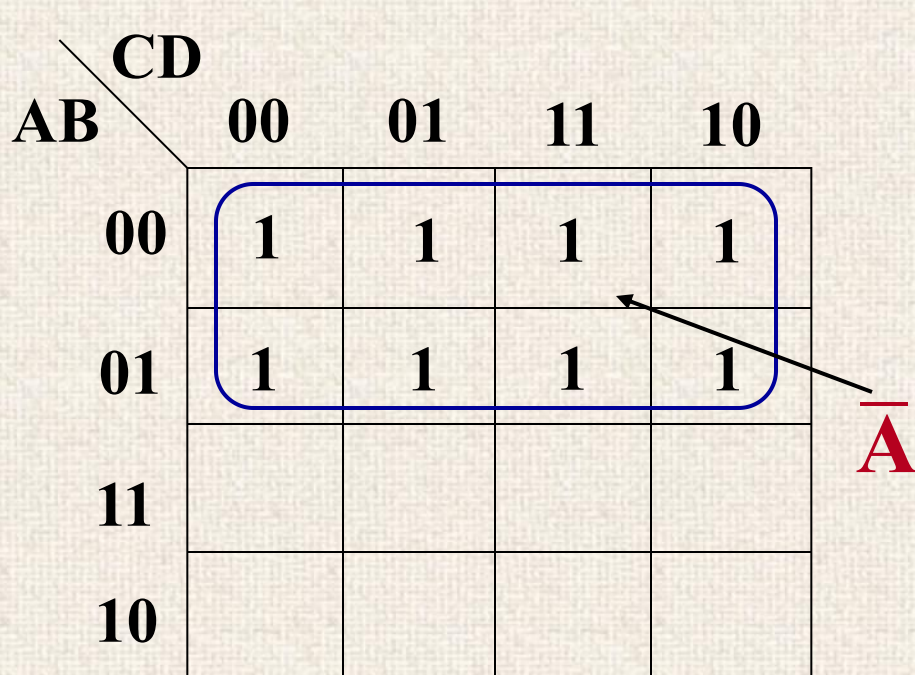
AB \ CD	CD			
	00	01	11	10
00	1	1	1	1
01			1	1
11			1	1
10				

AB \ CD	CD			
	00	01	11	10
00			1	1
01	1			1
11	1			1
10			1	1





(3) 卡诺图上任何八个标1的方格相邻，可以并为一项，并可消去三个变量。例：





思考题：

AB \ CD	CD			
	00	01	11	10
00		1	1	
01		1	1	
11		1	1	
10		1	1	

综上所述，在 n 个变量的卡诺图中，只有 2^i 个相邻的标1方格（必须排列成方形格或矩形格的形状）才能圈在一起，合并为一项，该项保留了原来各项中 $n-i$ 个相同的变量，消去 i 个不同变量。





4) 用卡诺图化简逻辑函数（化为最简与或式）

①最简标准：项数最少，意味着卡诺图中圈数最少；
每项中的变量数最少，意味着卡诺图中的圈尽可能大。

例 将 $F(A, B, C) = \sum m(3, 4, 5, 6, 7)$
化为最简与或式。





BC \ A	00	01	11	10
0			1	
1	1	1	1	1

$$F = A + BC \quad (\text{最简})$$

BC \ A	00	01	11	10
0			1	
1	1	1	1	1

$$F = A\bar{B} + BC + ABC\bar{C} \quad (\text{非最简})$$

$$\begin{aligned} & \{ = A(\bar{B} + B\bar{C}) + BC \\ & = A(\bar{B} + \bar{C}) + BC \\ & = A\bar{B}\bar{C} + BC \\ & = A + BC \} \end{aligned}$$

② 化简步骤 (结合举例说明)





例 将 $F(A,B,C,D) = \sum m(0,1,3,7,8,10,13)$ 化为最简与或式。

解: (1) 由表达式填卡诺图;

(2) 圈出孤立的标1方格;

m_{13}

		CD			
		00	01	11	10
AB	00	1	1	1	
	01			1	
	11		1		
	10	1			1





AB \ CD		CD				
		00	01	11	10	
00	1	1	1			$\bar{A}\bar{B}\bar{C}$ $\bar{A}CD$
01				1		
11			1			$AB\bar{C}D$
10	1				1	$A\bar{B}\bar{D}$

(3) 找出只被一个最大的圈所覆盖的标1方格, 并圈出覆盖该标1方格的最大圈; m_7, m_{10}

(4) 将剩余的相邻标1方格, 圈成尽可能少, 而且尽可能大的圈. m_0, m_1





(5) 将各个对应的乘积项相加, 写出最简与或式.

$$F(A,B,C,D)=AB\bar{C}D+\bar{A}CD+A\bar{B}\bar{D}+\bar{A}\bar{B}\bar{C}$$

例: $F(A,B,C,D)=\bar{A}\bar{C}+\bar{A}C\bar{D}+ABD+\bar{B}\bar{C}+\bar{B}C\bar{D}$

AB \ CD	CD			
	00	01	11	10
00	1	1		1
01	1	1		1
11		1	1	
10	1	1		1

$$F(A,B,C,D)=ABD+\bar{B}\bar{D}+\bar{A}\bar{D}+\bar{C}D$$





一种特殊情况:

AB \ C	0	1
00		1
01	1	1
11	1	
10	1	1

$$F = A\bar{B} + B\bar{C} + \bar{A}C$$

AB \ C	0	1
00		1
01	1	1
11	1	
10	1	1

$$F = \bar{A}B + \bar{B}C + A\bar{C}$$

得到两种化简结果, 也都是最简的。





③ 化简中注意的问题

- (1) 每一个标1的方格必须至少被圈一次;
- (2) 每个圈中包含的相邻小方格数, 必须为2的整数次幂;
- (3) 为了得到尽可能大的圈, 圈与圈之间可以重叠;

AB \ CD		CD			
		00	01	11	10
00	1	1		1	
01	1	1		1	
11		1	1		
10	1	1		1	





(4) 若某个圈中的所有标1方格, 已经完全被其它圈所覆盖, 则该圈为多余的.

例如:

蓝色的圈为多余的.

$$F = \bar{A}B\bar{C} + \bar{A}CD + A\bar{C}D + ABC + (BD)$$

AB \ CD		CD			
		00	01	11	10
00				1	
01	1		1	1	
11			1	1	1
10			1		





④ 用卡诺图求反函数的最简与或式

方法: 在卡诺图中合并标 0 方格, 可得到反函数的最简与或式.

例:

BC \ A	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$\bar{F} = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{A}\bar{C}$$





- 常利用该方法来求逻辑函数F的最简与或非式, 例如将上式F上 的非号移到右边, 就得到F的最简与或非表达式.

$$F = \overline{\overline{A}\overline{B} + \overline{B}\overline{C} + \overline{A}\overline{C}}$$





逻辑函数化简的技巧

- 对较为复杂的逻辑函数，可将函数分解成多个部分，先将每个部分分别填入各自的卡诺图中，然后通过卡诺图对应方格的运算，求出函数的卡诺图。
- 对卡诺图进行化简。





例：化简逻辑函数

$$F = (AB + \bar{A}C + BD) \oplus (ABCD + \bar{A}CD + BCD + \bar{B}C)$$

AB \ CD	CD			
	00	01	11	10
00		1	1	1
01			1	1
11	1	1	1	1
10		1	1	

$\oplus$

AB \ CD	CD			
	00	01	11	10
00			1	1
01			1	
11			1	
10		1	1	1

=

AB \ CD	CD			
	00	01	11	10
00		1		
01				1
11	1	1		1
10				1

$$F = \bar{A}BCD + ABC + BCD + ACD$$





5) 不完全确定的逻辑函数及其化简

在某些实际数字电路中, 逻辑函数的输出只和一部分最小项有**确定**对应关系, 而和余下的最小项无关. 余下的最小项无论写入逻辑函数式还是不写入逻辑函数式, 都不影响电路的逻辑功能. 把这些最小项称为**无关项**. 用英文字母d (don't care)表示, 对应的函数值记为“×”。

包含**无关项**的逻辑函数称为**不完全确定**的逻辑函数。





利用不完全确定的逻辑函数中的无关项往往可以将函数化得更简单.

例：设计一个奇偶判别电路. 电路输入为8421BCD码, 当输入为偶数时, 输出为 0 ;当电路输入为奇数时, 输出为1 .

由于8421BCD码中无1010~1111这6个码, 电路禁止输入这6个码. 这6个码对应的最小项为无关项.





真值表

A	B	C	D	F	A	B	C	D	F
0	0	0	0	0	1	0	0	0	0
0	0	0	1	1	1	0	0	1	1
0	0	1	0	0	1	0	1	0	×
0	0	1	1	1	1	0	1	1	×
0	1	0	0	0	1	1	0	0	×
0	1	0	1	1	1	1	0	1	×
0	1	1	0	0	1	1	1	0	×
0	1	1	1	1	1	1	1	1	×

$$F(A,B,C,D)=\sum m(1,3,5,7,9)+\sum d(10 \sim 15)$$





$$F(A,B,C,D)=\Sigma m(1,3,5,7,9)+\Sigma d(10 \sim 15)$$

若不利用无关项（即将卡诺图中的 $\times$ 均作0处理），则化简结果为：

$$F(A,B,C,D)=\bar{A}D+\bar{B}\bar{C}D$$

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	$\times$	$\times$	$\times$	$\times$
10	0	1	$\times$	$\times$

若利用无关项（即将卡诺图中的 $\times$ 按化简的需要任意处理，将有些 $\times$ 当作0，有些 $\times$ 当作1），则化简结果为：

$$F(A,B,C,D)=D$$





完整地将函数写为:

$$\begin{cases} F(A,B,C,D)=D \\ \Sigma d(10 \sim 15)=0 \end{cases}$$

再例:

$$F=(A \oplus B)C\bar{D} + \bar{A}B\bar{C} + \bar{A}\bar{C}D \quad \text{且 } AB+CD=0$$

AB \ CD	00	01	11	10
00	0	1	×	0
01	1	1	×	1
11	×	×	×	×
10	0	0	×	1

$$F(A,B,C,D)=B+\bar{A}D+AC$$





3. Q-M法（奎恩-麦克拉斯基）

Q-M法又称**系统简化法**，适用于任意多变量的函数，有较严格的算法，可借助于计算机解题。

系统简化法分三个步骤进行：

- (1) 求出函数的全部主要项；
- (2) 选出函数的必要项；
- (3) 选择主要项，使它与必要项一起包含给定函数的全部最小项，建立函数的最简与一或式。





注意:在无特殊说明的情况下,为使逻辑函数化的更简单,
均应按上述**第二种**方法(卡诺图化简)处理最小项.

